

ADVANCE PROBLEM PROGRAMMING

SLO 2: Implement Threading

1. Write a Java Program to Print "Hello" Using a Thread

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Hello");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Output:

Hello

2. Create Two Threads to Print Even and Odd Numbers Separately

```
class EvenThread extends Thread {  
    public void run() {  
        for (int i = 2; i <= 10; i += 2) {  
            System.out.println("Even: " + i);  
        }  
    }  
}
```

```
class OddThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 9; i += 2) {  
            System.out.println("Odd: " + i);  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        EvenThread even = new EvenThread();  
        OddThread odd = new OddThread();  
  
        even.start();  
        odd.start();  
    }  
}
```

Possible Output:

Even: 2

Odd: 1

Even: 4

Odd: 3

Even: 6

Odd: 5

Even: 8

Odd: 7

Even: 10

Odd: 9

The exact order may vary because both threads execute concurrently.

3. Demonstrate Thread Sleep and Join in Java

- `sleep()` pauses the current thread for a specified period.
- `join()` makes the current thread wait until another thread finishes.

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++) {  
            System.out.println("Thread: " + i);  
  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                System.out.println("Thread interrupted");  
            }  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();
```

```
t.start();

try {
    t.join();
} catch (InterruptedException e) {
    System.out.println("Main thread interrupted");
}

System.out.println("Main thread completed");
}
```

Output:

Thread: 1

Thread: 2

Thread: 3

Main thread completed

Here, `Thread.sleep(1000)` pauses the thread for approximately **1 second** after each iteration. The `join()` method ensures that the main thread waits until `t` completes its execution.